

Vietnam National University
Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering



BACHELOR THESIS

Android USB gadget driver fuzzer

Instructors:

Dr. Nguyen An Khuong HCMUT
Nguyen Le Thanh VNG Corp
Nguyen Quoc Bao
Nguyen Anh Khoa

Opponent:

Dr. Tran Tuan Anh HCMUT

Student:

Bui Quang Minh 1712145

Ho Chi Minh City, July 2021

Acknowledge

I want to thank my advisor, Dr. Nguyen An Khuong for guiding and giving me a lot of constructive feedback during this thesis despite a lot of difficulties in the pandemic. I want to thank Mr. Nguyen Le Thanh for giving me the best opportunity to complete this work. I want to thank my seniors, Mr. Nguyen Quoc Bao and Mr. Nguyen Anh Khoa for their guidance, listening and advice on many obstacles. I want to thank all EFIENS members and the security community for helping me in my learning journey. Most importantly, I want to thank my family for always being by my side and encouraging me during this work.

Abstract

Fuzzing has become one of the most popular software testing methods recently due to its efficiency in bug finding. However, there are still open problems to adapt fuzzing method to various softwares. One of which is USB gadget driver because of the hardware requirement and complexity to interact with. In this thesis, we propose a system to perform fuzzing testing on USB gadget drivers automatically. Our system utilizes the existing AFL fuzzer for test case generator and code coverage collecting capability of the operating system for an effective fuzzing performance. Moreover, our system is able to run in an emulated environment without the necessity of any special hardwares.

Contents

1	Introduction	1
1.1	Problem statement	1
1.2	Objectives	2
1.3	Structure	2
2	Background	3
2.1	Universal Serial Bus (USB)	3
2.1.1	Overall	3
2.1.2	USB stack in Linux kernel	4
2.2	Fuzzing	5
2.2.1	Definition	5
2.2.2	Fuzzer classification	6
2.2.3	Code coverage	6
2.2.4	Basic block	6
2.2.5	American Fuzzy Lop (AFL) fuzzer	7
2.2.6	USB gadget driver fuzzing approaches	9
2.2.7	Linux kernel code coverage	12
3	Related works	14
3.1	umap2	14
3.2	vUSBf	15
3.3	USBFuzz	15
3.4	Syzkaller	16
4	Proposed architectural design and implementation	18
4.1	Architectural design	18
4.2	Environment setup	19
4.3	Component implementation	20

4.3.1	Harness	20
4.3.2	Manager	21
4.3.3	Executor Driver	22
4.3.4	Executor	23
4.4	Code coverage collecting	25
4.4.1	AFL compatibility	25
4.4.2	Code coverage collecting interval	25
5	Evaluation	27
5.1	Execution speed	27
5.2	Code coverage determinism	28
6	Conclusion	29
6.1	Conclusion	29
6.2	Future work	29
	Appendices	30
A	User Interface	31

List of Figures

2.1	USB stack	4
2.2	A basic block of x86 instruction code	7
2.3	Code compiled with normal GCC	8
2.4	Code compiled with AFL GCC	8
2.5	An AFL fuzzer iteration	9
2.6	USB fuzzer using real hardware	10
2.7	USB fuzzer using emulated hardware	11
2.8	USB fuzzer with USB stack modification	12
2.9	Linux kernel code compiled with code coverage support	12
3.1	FaceDancer USB board	14
3.2	vUSBf architecture	15
3.3	USBFuzz architecture	16
3.4	Syzkaller architecture	17
4.1	System design	18
4.2	Communication state between Harness and Manager	21
4.3	Manager's work flow	22
4.4	Executor interaction with Executor Driver	24
A.1	Start-up process of our system	31
A.2	AFL fuzzer's UI	32

List of Tables

5.1	Execution speed	27
5.2	Stability scores during 1 hour testing	28

1.1 Problem statement

Universal Serial Bus (USB) is widely used in the connection between mobile phone and external devices. As almost everyone has a mobile phone nowadays, attackers put a lot of effort on finding new attack vectors to compromise victims' device and USB is one of the most promising one. An attack via USB requires a physical USB connection between attacker's malicious device and victim's device. A common scenario is that victims mistakenly plug their phones to a free charging socket which contains attacker's malicious code. A careless victim may grant extra permissions for the USB connection when being asked. Beside that, attacker can perform electrical attack which damages the victim's phone by providing high voltage output. More dangerously, attacker exploits the vulnerabilities in USB driver to fully compromise victim's phone since the driver code runs in privileged mode.

USB driver like other softwares is susceptible to programming bugs and vulnerabilities. One of the methods to minimize those mistakes is to do software testing intensively before releasing. As a software testing method, fuzzing has become a very effective technique in finding vulnerabilities with a lot of found bugs in various types of softwares [12][6]. However, due to the difficulty in interacting with USB driver and the hardware requirement, fuzzing in USB driver can be a big challenge. A lot of research has been conducted to make USB driver fuzzing more flexible and efficient such as vUSBf [15], USBFuzz [14], Syzkaller [7]. Nonetheless, these fuzzers only support USB host testing and ignore USB gadget side which is our mobile phones. It is understandable in the past as most USB attacks target USB host side (computer, laptop) but with the growth in mobile phone usage, more concerns need to be drawn to the USB gadget side testing.

For all the above reasons, in this thesis, we propose and implement an automated USB gadget driver fuzzer. Our system is able to run without any special USB hardwares so that USB gadget driver testing can become accessible to more people.

1.2 Objectives

In this thesis, we design and implement a system to perform security testing using fuzzing method on USB gadget driver. Our prototype supports USB gadget driver fuzzing in Linux kernel version 4.14 Long Term Support, this is the version upon which Android 9, 10, 11 is built [2]. Our fuzzer should have the following properties:

- Generate test cases automatically
- Spin up an isolated operating system for testing USB gadget driver
- Load the tested USB gadget driver and deliver generated test cases to it
- Monitor and display the result of testing process

With the above requirements, we

- Understand AFL fuzzer [12] and integrate it to our fuzzing system as a test case generator
- Understand the Linux kernel USB stack and learn to write a USB host driver
- Understand and customize the code coverage collecting system as we needed
- Build the isolated operating system for USB gadget driver testing
- Design and implement a test case delivery system

1.3 Structure

In Chapter 2, we present the background about fuzzing method and USB stack in the Linux kernel. In Chapter 3, we describe previous research works on USB driver fuzzing problem and their methodologies. In Chapter 4, we propose the architecture of our system and discuss the detailed implementation. In Chapter 5, we evaluate our system based on the execution speed and code coverage determinism. Finally, in Chapter 6, we give the conclusion and discussion about future work.

2.1 Universal Serial Bus (USB)

2.1.1 Overall

USB is the most popular method for connecting peripheral devices to host system. USB protocol provides a standard for host system to determine the correct driver to handle various types of external device. Nowadays, there are 4 USB generations: USB 1.x, USB 2.0, USB 3.x and USB4.

USB has an asymmetric protocol: host side and gadget/device side. Host side is usually our personal computer, laptop while the gadget side is USB mass storage, mouse device. Mobile phone can be in host side or gadget side depending on the connection context. In every transaction between the host side and gadget side, host side is always the initiating side.

There are 4 types of USB transfer:

- **Control transfer:** is used to transfer commands and information during enumeration phase when the USB device is plugged into host system. USB device provides USB descriptor information so that host system can choose a matching handling driver. Control transfer is guaranteed to be as fast as possible by the host system. Moreover, error detection, re-transmission and delivery guarantee are supported in control transfer.
- **Interrupt transfer:** is used by the device to get the attention from host system. However, the interrupt cannot be transferred asynchronously to host system as the host side must be the initiating side in every USB transaction. As a result, the USB device must wait for host system's poll to transfer the interrupt. Interrupt transfer is usually used to transfer mouse, keyboard click interrupt so it is guaranteed to have low latency, error detection support as well as re-transmission on error.
- **Isochronous transfer:** delivers periodic, continuous and time-sensitive data such as audio and video stream. Isochronous transfer is guaranteed to have bounded

latency, but no error detection or re-transmission is provided.

- **Bulk transfer:** is used to transfer large chunks of data such as printer and scanner data. This transfer has the lowest priority, bandwidth and latency is not guaranteed. Bandwidth is only allocated when it is not needed by other transfers. However, error detection is supported and delivery is guaranteed.

2.1.2 USB stack in Linux kernel

Linux kernel supports both host and gadget side of USB.

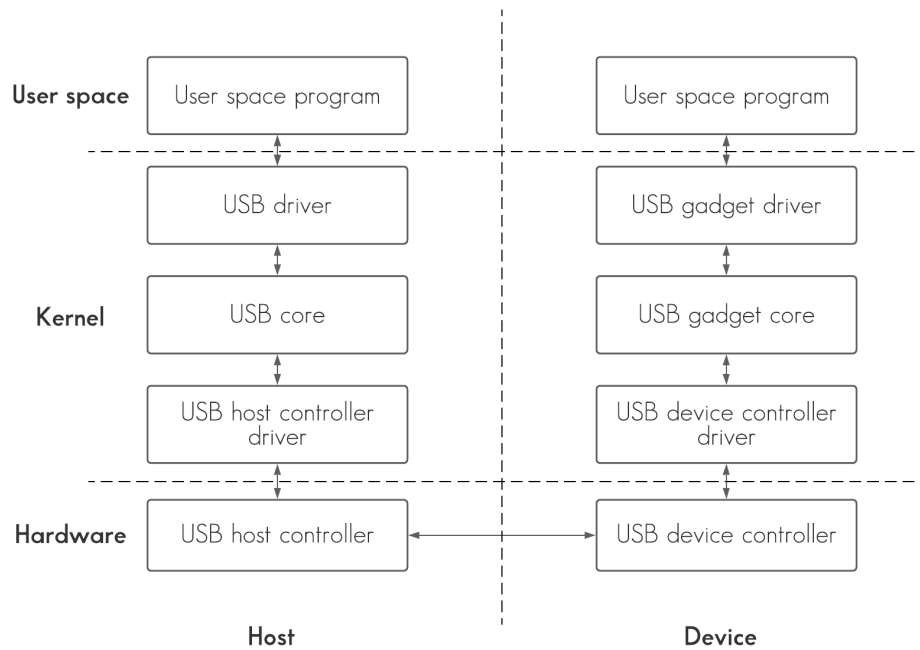


Figure 2.1: USB stack

USB host and gadget controller are connected in hardware layer via USB port.

USB host software layer starts with the USB host controller driver. This driver helps the operating system to communicate with the controller hardware. Above this layer is USB core, which contains primary USB functionalities. USB core also provides an interface for USB drivers to interact with USB host controller drivers without worrying about different types of USB host controllers. When a USB gadget is plugged into the host system, USB core plays a main role in looking for the correct USB driver for that USB gadget. At the next layer, USB driver provides USB gadget supported functionalities to user.

USB gadget software layer has the same structure as USB host one. It starts with a USB device controller driver to communicate with below USB device controller hardware. Next, USB gadget core provides the interface for USB gadget drivers to access and

control USB device controller driver. Lastly, USB gadget driver implement gadget side functionalities.

2.2 Fuzzing

2.2.1 Definition

Fuzzing is a software testing method in which a tested program will be run repeatedly with random, unexpected inputs. That program is then monitored to detect requirements violations such as program crash, time limit exceeding. Fuzzing, like all other software testing methods, tries to exercise as many program states as possible to minimize the occurrences of unexpected program states when releasing software.

A software testing program using fuzzing method is called a **fuzzer**.

A typical fuzzing algorithm is as follow

Algorithm 1 Fuzzing algorithm

Input: Seed test cases S

Output: Violation test cases V

T = S

repeat

 t = GenerateTestcase(T)

 o = Program(t)

if IsViolation(o) **then**

 add t to V

end if

if IsInteresting(o) **then**

 add t to T

end if

until User interruption **or** stop condition

return V

- T is called a **corpus** which is a set of good test cases that are kept to become input for next test case generation phase
- **GenerateTestcase** is a test case generation function
- o is the information about program state when running a test case
- **IsViolation** checks whether the program state information o violates requirements
- **IsInteresting** uses predefined heuristics to evaluate test case's quality based on information o

2.2.2 Fuzzer classification

Fuzzer is usually classified according to its GenerateTestcase stage and how it monitors, gets insight information about tested program.

On GenerateTestcase stage, we have 2 classes of fuzzer:

- **Mutation-based fuzzer:** uses genetic algorithm with the inputs from corpus set to generate a new test case with the hope that the new test case does not only inherit good properties from parent test cases but also has new better ones.
- **Generation-based fuzzer:** generates a new test case from scratch using predefined rules or grammar. Its main advantage is that it has a better chance to discover a new path which is completely unrelated to old test cases.

Modern fuzzers usually combine both of these methods with a heuristic ratio to maximize their advantages.

When it comes to how fuzzer uses the information from tested program, we can define 3 classes of fuzzer:

- **Black-box fuzzer:** generates test cases randomly not based on any insight information about program state of the tested software. This fuzzer can run very fast but due to its lack of insight information, it depends a lot on predefined grammar to generate test cases that can exercise a great amount of program states.
- **White-box fuzzer:** uses program analysis methods to collect every information about the program state combined with a constraint solver to generate test cases that can reach complex state [5][1]. However, this method can cause unacceptable overhead in the testing process.
- **Grey-box fuzzer:** stands in the middle of 2 previous fuzzers. It accepts the incompleteness of information for reducing computing overhead. The most well-known example is AFL fuzzer [12] that uses code coverage to approximate program state which incurs much lower overhead than full program state collecting.

2.2.3 Code coverage

Code coverage is the proportion of code that has been executed during the testing process. The control flow information about the tested software can be observed by collecting code coverage. Code coverage is widely used as a feedback for fuzzers [12][11][7] due to its compactness and decent approximation of program state.

2.2.4 Basic block

A basic block is a sequence of code instructions that whenever an instruction in the basic block is executed, any other instructions are guaranteed to be executed. In a different way, it means a basic block has only one entry which is its first code instruction and one

exit, its last code instruction. With that property, when we collect code coverage, we don't need to record the address of all instructions but only the address of one instruction per basic block.

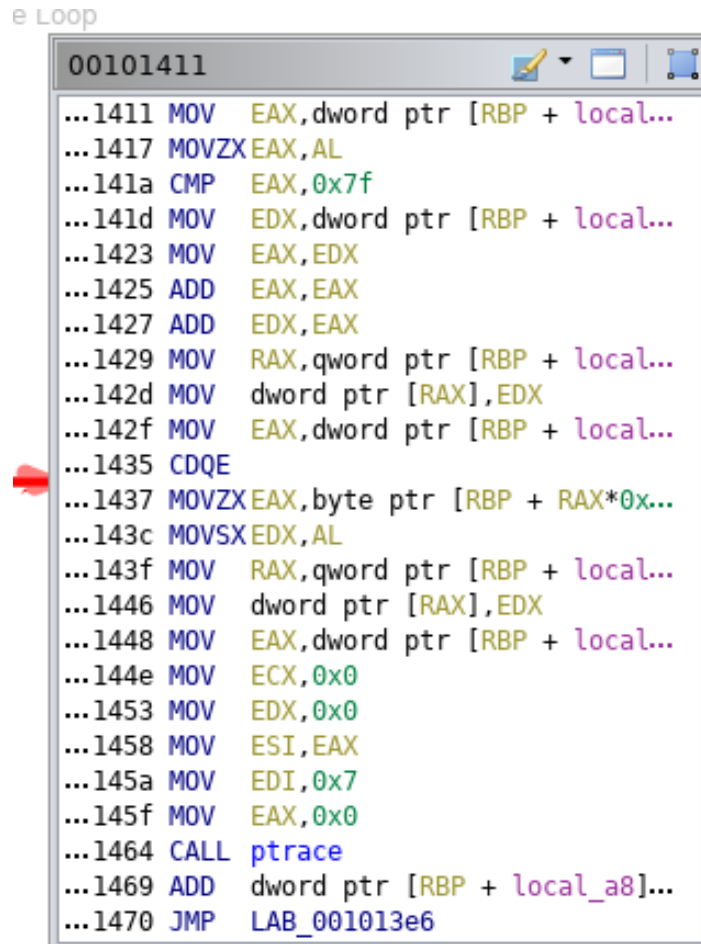
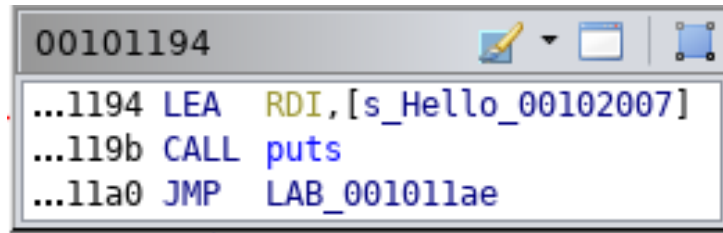


Figure 2.2: A basic block of x86 instruction code

2.2.5 American Fuzzy Lop (AFL) fuzzer

AFL fuzzer is a popular grey-box fuzzer that has been used in a lot of academic research [14][4]. It uses code coverage as an insight information of the tested program, code coverage is used to determine whether a test case is interesting. An interesting test case is a test case which has a newly observed code coverage. In order to get code coverage information, AFL fuzzer comes with a modified GNU compiler collection (GCC) to add a small code snippet to each basic block. These code snippets contain a call to `_afl_maybe_log` function that will mark the containing basic block as being executed when being called.

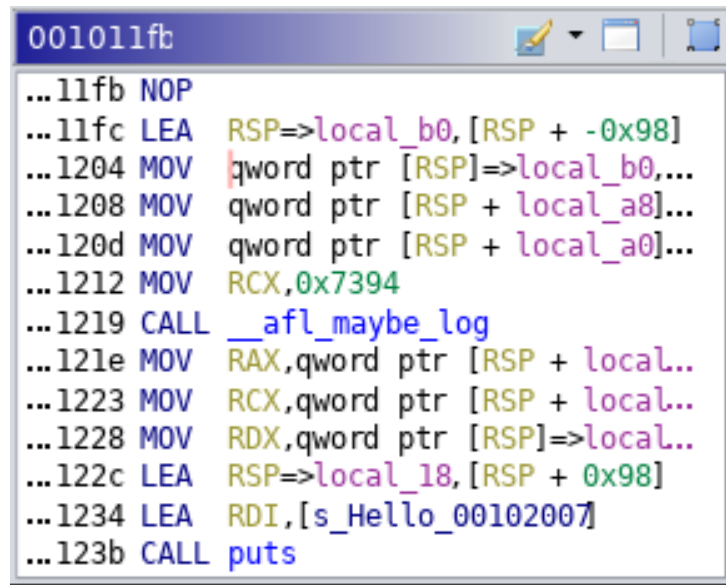


```

00101194
...1194 LEA RDI,[s_Hello_00102007]
...119b CALL puts
...11a0 JMP LAB_001011ae

```

Figure 2.3: Code compiled with normal GCC



```

001011fb
...11fb NOP
...11fc LEA RSP=>local_b0,[RSP + -0x98]
...1204 MOV qword ptr [RSP=>local_b0,...
...1208 MOV qword ptr [RSP + local_a8]...
...120d MOV qword ptr [RSP + local_a0]...
...1212 MOV RCX,0x7394
...1219 CALL __afl_maybe_log
...121e MOV RAX,qword ptr [RSP + local..
...1223 MOV RCX,qword ptr [RSP + local..
...1228 MOV RDX,qword ptr [RSP=>local..
...122c LEA RSP=>local_18,[RSP + 0x98]
...1234 LEA RDI,[s_Hello_00102007]
...123b CALL puts

```

Figure 2.4: Code compiled with AFL GCC

AFL fuzzer does not use a naive basic block coverage approach as this will miss some insight information about execution flow (e.g. with the execution flow $A \rightarrow B \rightarrow C$, this approach records only the information that block A, B, C has been executed which cannot differentiate with execution flow $A \rightarrow C \rightarrow B$). Instead, AFL fuzzer uses edge coverage which records the edge between 2 basic blocks and its direction. Moreover, AFL fuzzer also records the number of times an edge has been executed.

In order to store edge coverage information, AFL uses a 64kB array, each byte in the array represents the number of times an edge has been executed. AFL fuzzer uses shared memory mechanism for allocating this array between AFL process and tested process. The detailed implementation is as follow: AFL fuzzer allocates a 64kB shared memory region then passes the shared memory identifier to `__AFL_SHM_ID` environment variable; this environment variable is inherited by tested process when being forked by AFL fuzzer; the first call to `__afl_maybe_log` in the tested process will set up the shared memory in that process. After finishing setting up shared memory, `__afl_maybe_log` and later calls will compute and write code coverage information to that memory region.

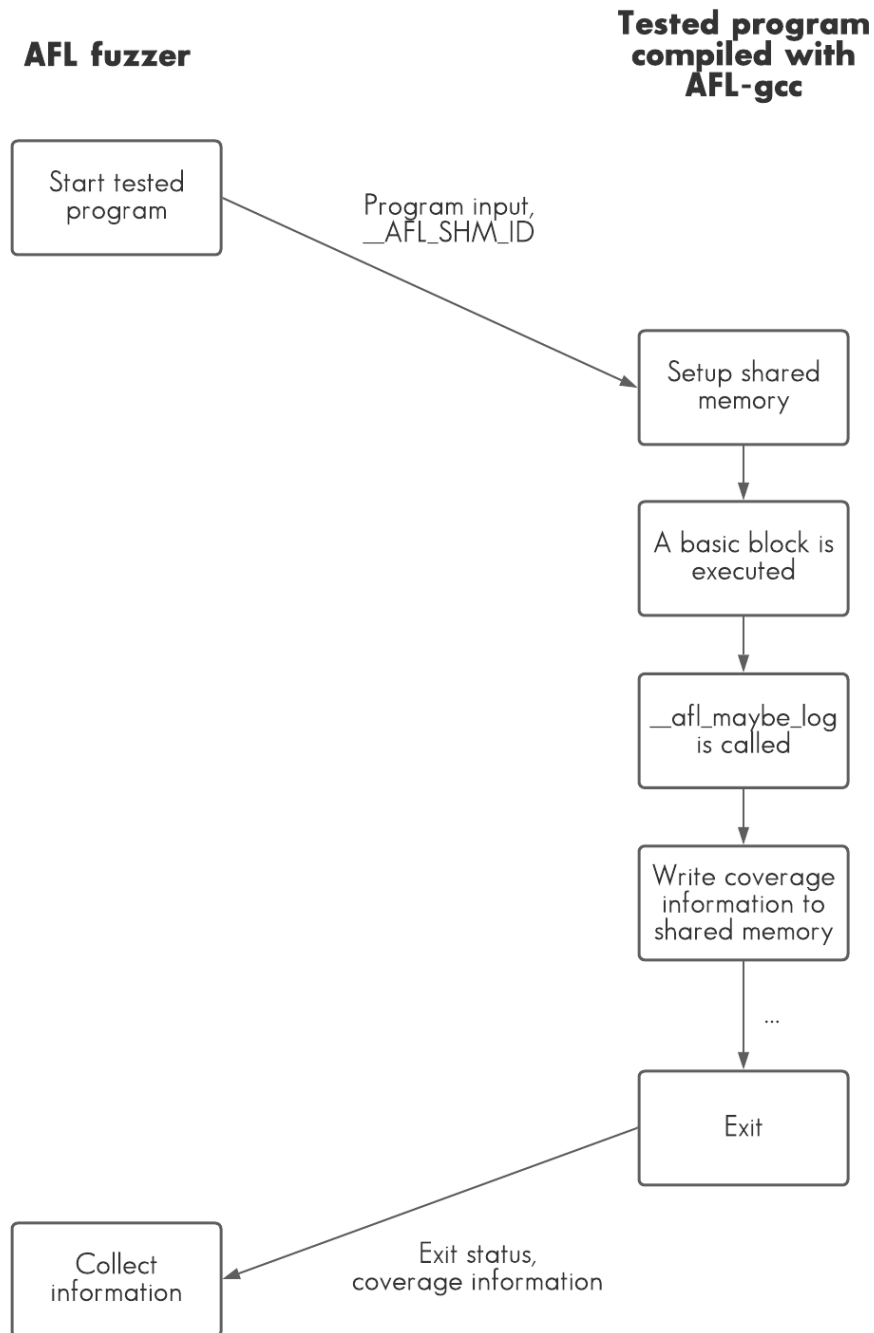


Figure 2.5: An AFL fuzzer iteration

2.2.6 USB gadget driver fuzzing approaches

The biggest challenge of USB gadget driver fuzzing is how to deliver test cases to the gadget driver's code. We will discuss some possible approaches for this problem.

(a) Using real hardware device

With this approach, we plug the real USB gadget device that contains the target USB gadget driver to our host system. On the host, a fuzzer generates test cases and these test cases are delivered through normal hardware connection. This approach simulates the real use case which makes the result more reliable. However, it is very hard to scale up when using this approach, scaling up means more hardware and more USB port connection. These hardware costs may raise the overall software testing cost too high. Furthermore, it is very difficult to automate the plugging/unplugging USB gadget operation.

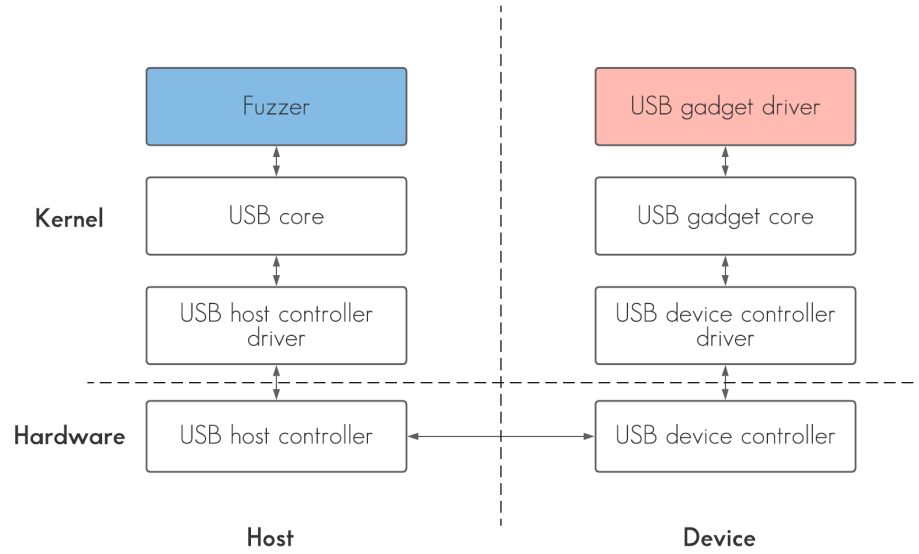


Figure 2.6: USB fuzzer using real hardware

(b) Emulating the USB connection with QEMU

This approach uses QEMU to emulate a USB gadget device. Our fuzzer is connected to the emulated hardware to deliver test cases to the tested USB gadget driver. This solution helps to resolve the scaling problem of the previous approach as it is easier to create more tested instances. Besides, plugging/unplugging USB operations can be automated at the software level. However, the main disadvantage of this approach is that QEMU currently does not support emulating USB device controllers. We need to implement an emulated USB device controller on our own which requires extensive research on hypervisor and emulator topic, especially about QEMU software.

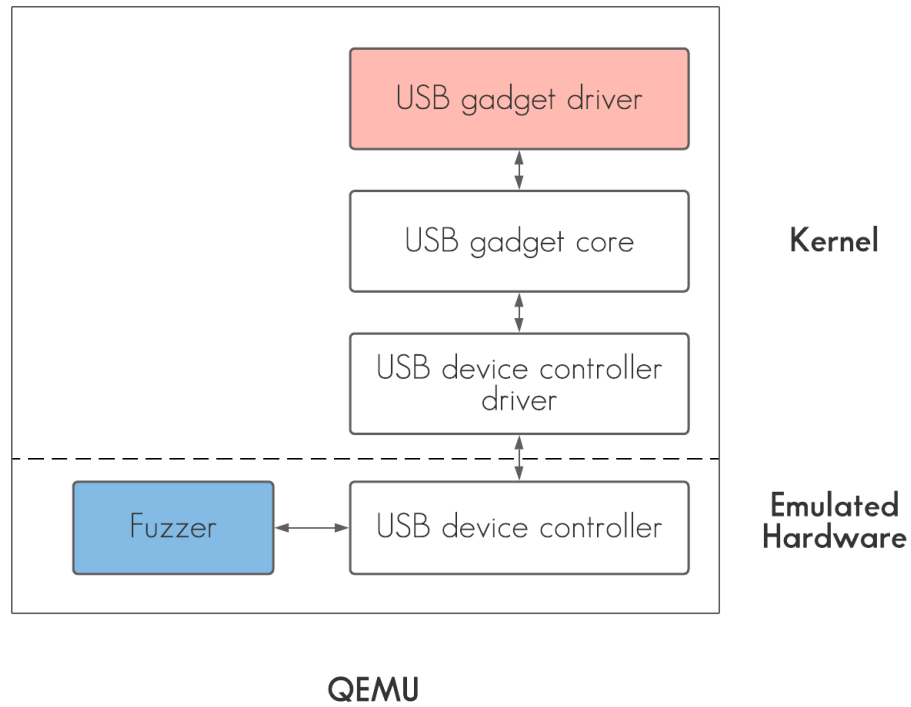


Figure 2.7: USB fuzzer using emulated hardware

(c) USB stack modification

The Linux kernel supports `dummy_hcd` feature as a way to emulate USB connection right inside the kernel without the need of hardware. With this feature, new USB host and gadget controller drivers are created, they are connected to each other at the software level, no USB controller or USB port is needed. This solution has all advantages of the second approach and it does not need any emulator/hypervisor modifications.

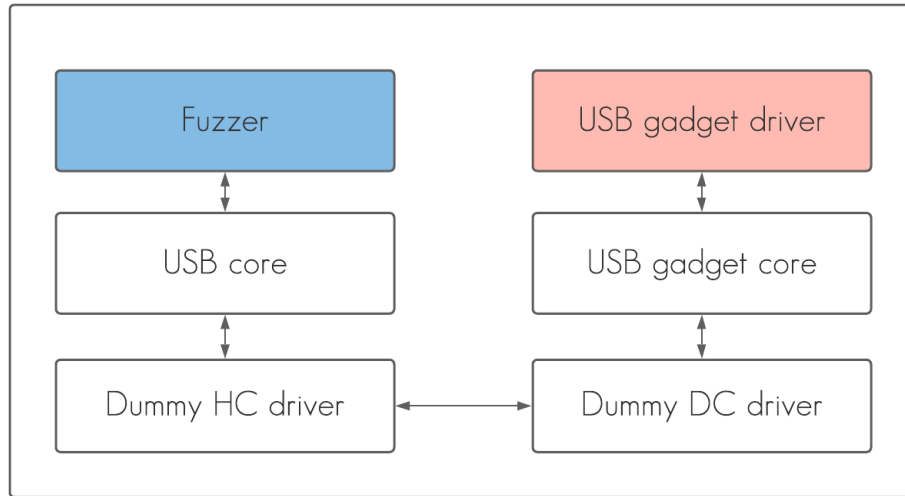


Figure 2.8: USB fuzzer with USB stack modification

2.2.7 Linux kernel code coverage

Linux kernel currently uses `-fsanitize=trace-pc` instrumentation option in GNU Compiler Collection (GCC) to add a function call to `__sanitizer_cov_trace_pc` into every basic block [3].

```

0010011c - LAB_0010011c
LAB_0010011c
...011c ADD    RBX, RBP
...011f SHR    RBX, 0xc
...0123 MOV    RBP, RBX
...0126 CALL   __sanitizer_cov_trace_pc
...012b SHL    RBP, 0x6
...012f ADD    RBP, qword ptr [vmemmap_bas...
...0136 MOV    R15, qword ptr [RBP + 0x20]
...013a TEST   R15B, 0x1
...013e JNZ    LAB_0010019c

```

Figure 2.9: Linux kernel code compiled with code coverage support

This code coverage support is mainly used for syzkaller fuzzer [7], which is originally a syscall¹ fuzzer. Its primary mission is to test the syscall handler in Linux kernel which is in process context². Therefore, in the version 4.14 which is used in this thesis, code

¹**syscall** (System call): is the most traditional and most obvious interaction point between user space and the kernel [10]

²**Process context**: is the state when kernel handles syscall requests from the user space program.

coverage feature only supports process context not interrupt context³. However, USB handler can happen in interrupt context. In order to tackle this problem, we propose a patch to Linux kernel which is discussed in Section 4.4.

³**Interrupt context:** Interrupt context handles interrupts that happen asynchronously caused by hardware or software (e.g. a mouse click sends an hardware interrupt to the CPU)

Related works

In this chapter, we describe previous research on USB host driver fuzzing and their methodologies.

3.1 umap2

umap2 is an open source project developed by NCC Group for USB host fuzzing [9]. It uses Facedancer programmable USB board for USB test case injection and delivery. Its main advantage is that the data flow in testing process is the same as normal real world connection, which makes the testing result more reliable. However, as discussed in Section 2.2.6, hardware-based approach is hard to scale up and automate. Furthermore, umap2 is a black-box fuzzer that collects no insight information about tested USB host. As a result, it becomes very inefficient.



Figure 3.1: FaceDancer USB board
Source: <https://int3.cc/products/facedancer21>

3.2 vUSBf

vUSBf is a USB host driver fuzzer which was published at Black Hat Europe 2014 [15]. vUSBf uses QEMU to emulate the OS as well as USB host drivers. It uses QEMU supported protocol, USB redirection protocol, which helps to deliver USB test cases through TCP, UDP or Unix socket. The main drawback of this fuzzer is its inefficiency as a black-box fuzzer.

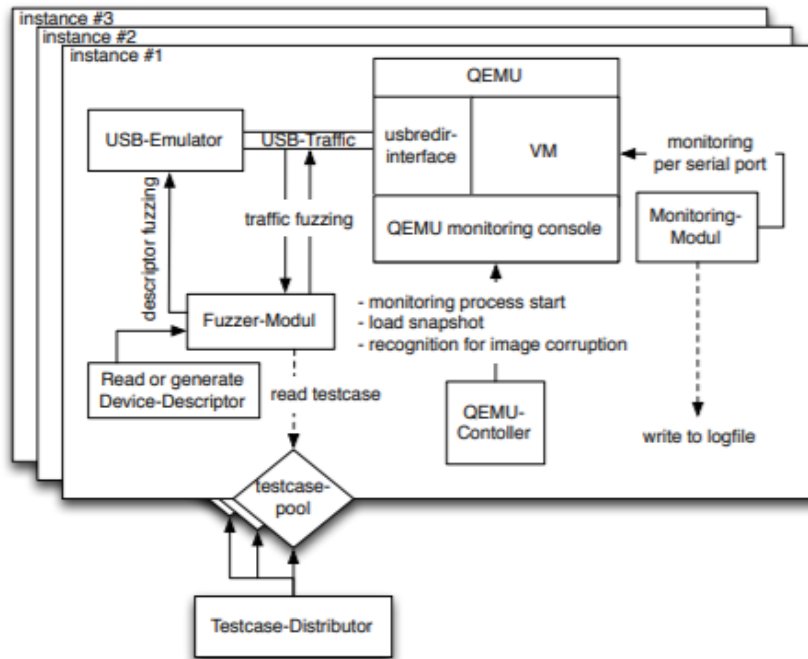


Figure 3.2: vUSBf architecture

Source: <https://www.blackhat.com/docs/eu-14/materials/eu-14-Schumilo-Dont-Trust-Your-USB-How-To-Find-Bugs-In-USB-Device-Drivers-wp.pdf>

3.3 USBFuzz

USBFuzz is a USB host driver fuzzer which was publicized in 2020. It uses the QEMU emulated host controller for emulating USB connection instead of using usbredir protocol like vUSBf. Besides, it also utilizes the code coverage collecting capability in Linux kernel to boost the fuzzing's efficiency. With a lot of improvements compared to vUSBf, it discovered 47 unique bugs during the evaluation process [14].

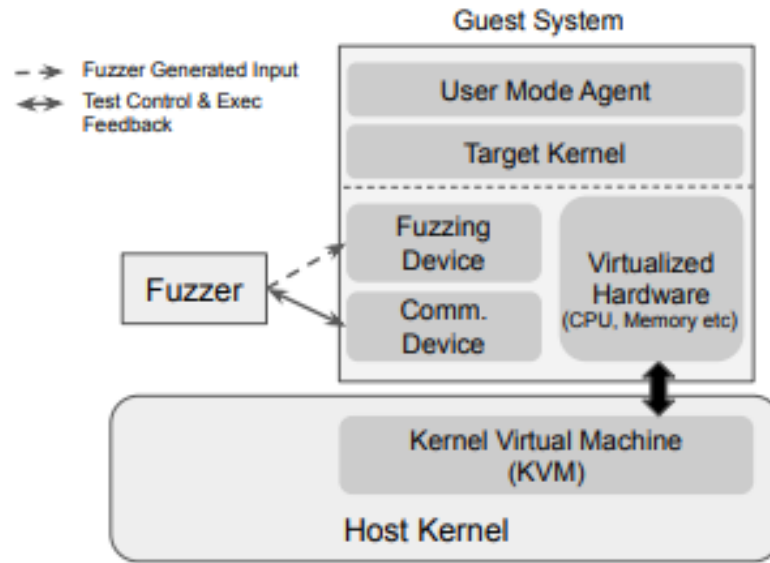


Figure 3.3: USBFuzz architecture

Source: <https://hexhive.epfl.ch/publications/files/20SEC3.pdf>

3.4 Syzkaller

Syzkaller is a syscall fuzzer¹ developed by Google engineers [7]. Syzkaller supports syscall fuzzing on Linux, OpenBSD and even Windows.

Syzkaller uses predefined grammar and code coverage collected from kernel to generate test cases, which has been proved to be very effective with thousands of found bugs. At the moment, Syzkaller is run on Google infrastructure, every found bug will be reported to maintainers by email. Online dashboard of Syzkaller can be found at <https://syzkaller.appspot.com/upstream>.

¹Syscall fuzzer: generates system calls to test kernel handler

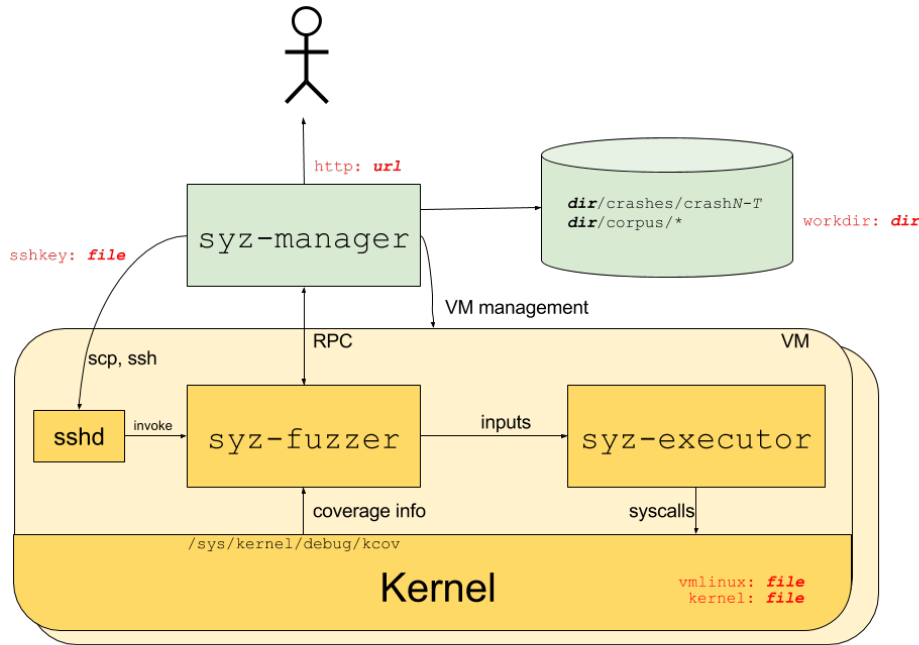


Figure 3.4: Syzkaller architecture

Source: <https://github.com/google/syzkaller/blob/master/docs/internals.md>

In 2017, Syzkaller started to support USB host driver fuzzing in Linux kernel. It uses `dummy_hcd` to emulate USB controller and have them connect on the software layer. Some pseudo syscalls are created to adapt the existing syscall fuzzer to USB subsystem. USB test cases are provided as the arguments of those pseudo syscalls and delivered through the modified USB stack to the target driver. With this approach, Syzkaller has found more than 300 bugs in USB subsystem [8]. However, due to some Linux specific implementations, porting this USB fuzzer to other OSes can be a big challenge.

All the above research focuses only on USB host driver fuzzing without support for USB gadget driver. In the next chapter, we propose our methodology for performing fuzzing on USB gadget driver.

Proposed architectural design and implementation

4.1 Architectural design

Among fuzzing approaches stated in Chapter 2, we decide to choose the USB stack modification approach since it scales easily when our computing power increases. Moreover, with the hardware abstraction from the Linux kernel, we don't need to give much care about hardware related implementation like the QEMU modification approach.

The architectural design is shown in Figure 4.1

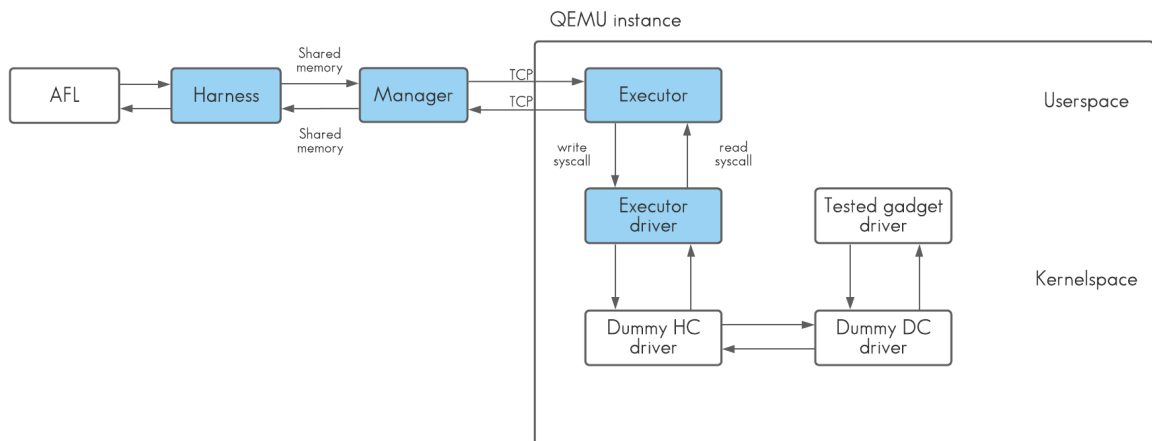


Figure 4.1: System design

- **AFL fuzzer** is used as a test case auto-generator for our system
- **Harness** emulates normal AFL fuzzer's tested program
- **Manager** starts up QEMU instance, sets up shared memory for communicating with **Harness** and sets up a TCP server

- **Executor** connects to manager, monitors kernel log, communicates with **Executor Driver**
- **Executor Driver** exposes a device for **Executor** communication, delivers USB test cases and gets code coverage information

Here is the overall execution flow of the system:

1. User runs the Manager with information about tested gadget driver
2. Manager spins up the QEMU instance
 - Executor is started, it compiles and loads Executor Driver with the information provided by Manager
 - Tested gadget driver and required kernel modules are loaded
 - Executor connects to Manager and waits for test cases
3. Manager starts the AFL fuzzer and receives the generated test case from Harness
4. Manager sends the test case to Executor
5. Executor receives the test case, sends it to Executor Driver and monitors kernel log
6. Executor Driver sends the test case to tested gadget driver using USB core API
7. Tested USB gadget driver receives the test case, executes it
8. Executor collects code coverage information or crash information and send it back to Manager
9. Manager receives the result and send it to AFL fuzzer
10. AFL fuzzer analyzes the result and generates a new test case, sends it to Manager, the test case flow loops to step 4 until user's stop signal

4.2 Environment setup

The following steps is how we set up our fuzzing environment system:

1. Install QEMU software and required libraries for compiling Linux kernel

```
1 apt-get install qemu-system build-essential libncurses-dev  
2 libssl-dev bison flex
```

2. Compile Linux kernel image

- Download Linux kernel source code from <https://www.kernel.org/>. In this thesis, we use Linux kernel version 4.14 Long Term Support as it is also one of the versions from which Google chooses to implement Android [2].
- Configure Linux kernel using default configuration with some modifications as follow

```

1      # For dummy_hcd use
2      CONFIG_USB_DUMMY_HCD=m
3      # Turn on USB gadget stack
4      CONFIG_USB_GADGET=y
5      CONFIG_USB_LIBCOMPOSITE=m
6      # For code coverage
7      CONFIG_KCOV=y
8      # We only collect code coverage from our tested
   gadget driver
9      CONFIG_KCOV_INSTRUMENT_ALL=n
10     # Memory bug detector1
11     CONFIG_KASAN=y
12     CONFIG_KASAN_INLINE=y
13

```

- Apply the patch that will be discuss in Section 4.4 to Linux kernel source code
- Compile Linux kernel image

3. Create initial ramdisk

- Use `debootstrap` to create a Debian root directory
- Create a disk and copy the created root directory to the disk

4.3 Component implementation

4.3.1 Harness

As discussed in Section 2.2.5, in each iteration, AFL fuzzer provides program input, shared memory identifier and expects to receive exit status, code coverage information in shared memory of the tested program. To simulate this fuzzing flow, we implement a lightweight user space program called Harness.

The communication between Harness and AFL fuzzer is pretty simple. Harness receives the generated test cases from AFL fuzzer via standard input but does not set up the shared memory for coverage information collecting like a normal tested program. Instead, that shared memory allocation mission is given to the Manager. Then, when the Manager receives code coverage information from the QEMU instance, it can write that information directly to shared memory without the need of passing the information through the

¹Memory bug detector: is an early detector for memory corruption bugs even though these bugs have not caused crash in our system yet

Harness process. This way, we can reduce a lot of unnecessary data copying operations and inter-process communication overhead.

The communication between Harness and Manager is implemented by using shared memory. This shared memory is allocated by Manager, the shared memory identifier is transferred to AFL fuzzer when Manager starts it. Later, this identifier is provided to Harness via program argument by AFL fuzzer. The communication between Harness and Manager is modeled into a state machine with 6 states: `SHMID_START`, `SHMID_END`, `TESTCASE_START`, `TESTCASE_END`, `RESULT_START` and `RESULT_END`. The first fuzzing iteration starts with `SHMID_START` state, when Harness finishes writing the code coverage shared memory identifier provided by AFL fuzzer to Harness-Manager shared memory, it sets the state to `SHMID_END`. After seeing the state turning into `SHMID_END`, Manager starts reading that shared memory identifier and sets the state to `TESTCASE_START` when finishing reading. Next, Harness writes the test case it received from AFL fuzzer to Harness-Fuzzer shared memory and the state transitions into `TESTCASE_END`. Following that, Manager reads the test case, transfers it to QEMU instance and waits for the test case execution result. When the result arrives, Manager writes it to Harness-Manager shared memory, the state is set to `RESULT_START`. Later, Harness reads that result, sets state to `RESULT_END`, exits successfully if the test case does not trigger bugs, otherwise, it simulates a bug crash. The next iteration begins in `TESTCASE_START` state since the Manager has already known the coverage shared memory identifier allocated by AFL fuzzer which does not change during the fuzzing process.

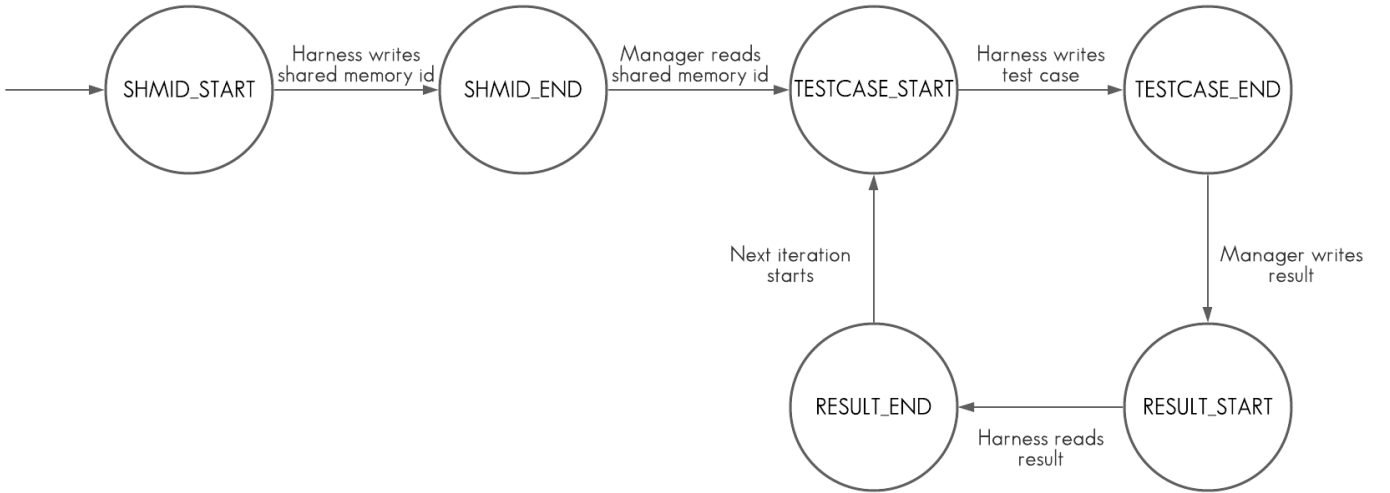


Figure 4.2: Communication state between Harness and Manager

4.3.2 Manager

Manager is a bridge between AFL fuzzer and the system inside QEMU instance. First, the Manager spins up the QEMU instance, then waits for the connection from Executor

process inside QEMU instance. After that, information about the tested USB gadget driver is transferred to Executor. Simultaneously, Manager starts the AFL fuzzer for test cases generating. When the start up phase is completed, Manager simply becomes a forwarder for the communication between AFL fuzzer and QEMU instance.

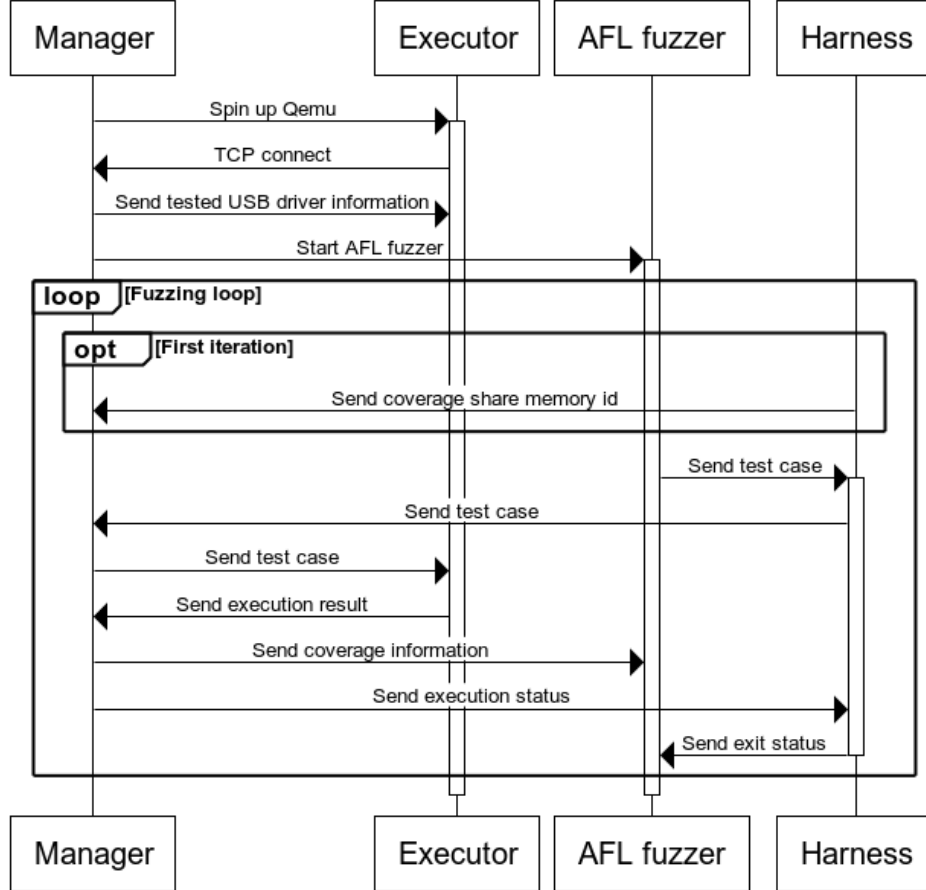


Figure 4.3: Manager's work flow

4.3.3 Executor Driver

Executor Driver plays a role as a USB host driver in our system. It is compiled with the `idVendor` and `idProduct` provided by the Executor. These identifiers are matched with the tested USB gadget driver so that Executor Driver will be loaded when plugging the test USB gadget.

Executor Driver exposes a device for communication with the user space program (Executor). This device implements 5 functions: `open`, `release`, `write`, `ioctl` and `mmap`.

(a) `open`

`open` function is called when Executor opens the exposed device. This function will set up the inner `file` structure that is required by the kernel.

(b) release

release function is called when the reference counter of inner **file** structure is 0 (nothing references to **file** structure anymore). This function will deallocate resources back to the kernel.

(c) write

write function is used to transfer test cases from the Executor to Executor Driver then Executor Driver can deliver them to the tested USB gadget driver. This function supports 3 different types of testing USB packets: control, bulk and interrupt.

(d) mmap

To help Executor get the test case's status as well as code coverage information, Executor Driver implements **mmap** function instead of **read** function. Performance is the main reason for this decision. With **mmap**, a virtual memory region in Executor will be mapped to the same physical pages as the kernel virtual memory used for code coverage storage. So when Executor needs to get the code coverage information, it can read directly from mapped region instead of calling **read** function which costs a memory copying operation from kernel virtual memory to Executor virtual memory.

(e) ioctl (IO control)

This function support 2 commands: **FUZZER_ENABLE_KCOV** and **FUZZER_DISABLE_KCOV** which are used to notify the kernel to enable/disable kernel code coverage collecting. These calls will get blocked until code coverage collecting is enabled/disabled.

4.3.4 Executor

Executor has 3 main roles:

- Communicating with the Manager to receive test cases and send back code coverage, execution results.
- Communicating with the Executor Driver for test cases, code coverage transfer and controlling the code coverage collecting process.
- Monitoring kernel log for bug detection.

Executor Driver uses TCP protocol for communicating with the Manager. The interaction flow is described in Section 4.3.2 and summarized in Figure 4.3.

In order to communicate with the Executor Driver, Executor interacts with the driver's exposed device. First, the Executor calls **open** on that device, then uses **mmap** to set up code coverage shared memory. After that, it enters a loop cycle by requesting code coverage collecting via **ioctl** command **FUZZER_ENABLE_KCOV**. Test case is then transferred via **write** syscall. At the end of an iteration, **ioctl** is called with command **FUZZER_DISABLE_KCOV** to notify the kernel to turn off the code coverage collecting and code coverage information is then read from shared memory. That fuzzing loop continues until a stop

signal happens. At that time, shared memory is deallocated with `munmap` call and the exposed device is closed.

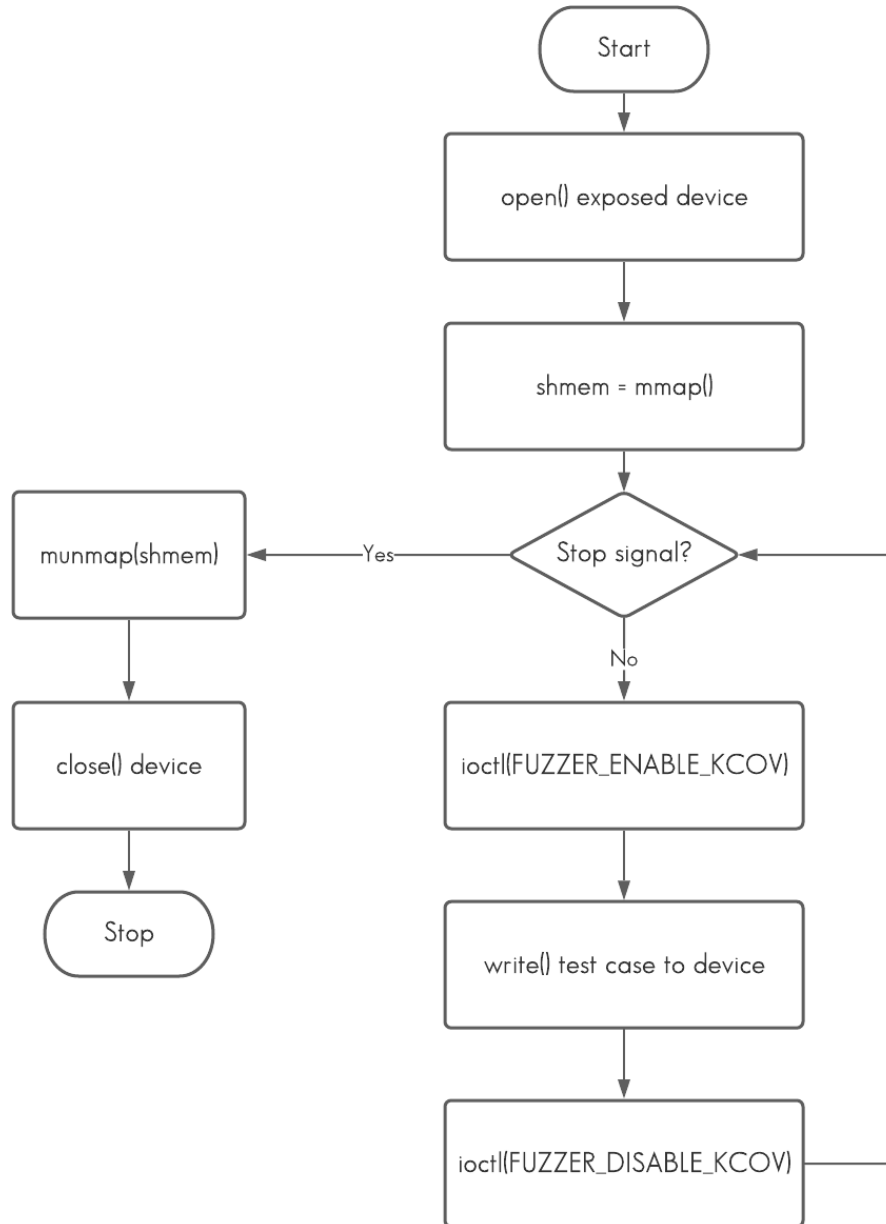


Figure 4.4: Executor interaction with Executor Driver

The kernel log is monitored via `/dev/kmsg`, our Executor uses the list of keywords from USBFuzz [14] to detect that bugs have happened.

```

1  "BUG : ",
2  "WARNING : ",

```



```

3  "INFO:",
4  "Unable to handle kernel paging request",
5  "general protection fault:",
6  "Kernel panic",
7  "PANIC: double fault",
8  "kernel BUG",
9  "BUG kmalloc-",
10 "divide error:",
11 "invalid opcode:"
12 "UBSAN:",
13 "unregister_netdevice: waiting for",
14 "trusty: panic",
15 "Call Trace:"

```

4.4 Code coverage collecting

4.4.1 AFL compatibility

As discussed in Section 2.2.5, AFL fuzzer uses a 64kB array to store edge code coverage, in which each byte represents the number of times that edge has been executed during a testing iteration. And in Section 2.2.7, we know that Linux kernel supports code coverage collecting by adding a call to function `__sanitizer_cov_trace_pc` to each basic block. So our mission is to implement function `__sanitizer_cov_trace_pc` to collect the code coverage information compatible with the AFL fuzzer.

The algorithm we implement is the same as one used in AFL fuzzer [13]

```

1  curr = hash_long(ip, 16);
2  coverage[curr ^ prev]++;
3  prev = curr >> 1;

```

`coverage` is a 64kB array to store code coverage, `curr`, `prev` is 16-bit unsigned integers which store current/previous basic block identifiers, `ip` is the instruction address of the instruction right after the call to `__sanitizer_cov_trace_pc` in that basic block. The basic block identifier is calculated by using built-in function `hash_long` which returns a 16-bit hash from `ip`. An edge index in coverage array is the result of exclusive or (XOR) operation on current basic block identifier and previous basic block identifier. Due to the commutativity of XOR operation, an edge from basic block A to basic block B is the same as an edge from basic block B to basic block A (have the same index in coverage array). However, we want to differentiate between these 2 edges. In order to achieve that, a basic block has 2 identifiers depending on whether it is the destination or the source of an edge. If a basic block is the destination of an edge the identifier is calculated as `hash_long(ip, 16)`, otherwise it is calculated as `hash_long(ip, 16) >> 1`.

4.4.2 Code coverage collecting interval

One of the biggest problems we face is to determine when to start and stop code coverage collecting in an iteration. Since code coverage is the main feedback in our

fuzzer, the correctness of it plays an important role in our testing system. As stated in Section 4.3.3, `ioctl` commands are used to deliver user space program's requests for code coverage collecting and when these `ioctl` calls return, it is guaranteed that the requests has been satisfied.

In our first implementation, when handling `FUZZER_ENABLE_KCOV` `ioctl` command, our Executor Driver immediately enables code coverage collecting. Whenever the Executor wants to disable code coverage collecting, it searches for test case execution completion message in the kernel log and issues a `FUZZER_DISABLE_KCOV` `ioctl` command when finding out the message. This message is written by the completion callback of the sending USB packet. While evaluating this implementation, we observe unacceptable code coverage determinism² which indicates our code coverage collecting interval is not deterministic or the testing environment is not the same in every iteration. Reading the Linux kernel documentation more carefully, we realize that the completion callback of the sending USB packet is called when the host side finishes processing the USB packet, it bears no relationship with the execution on the gadget side. Therefore, that callback neither indicates the completion of test case execution nor synchronizes with it. As a result, the code coverage collected is indeterministic. Besides, we also realize the code coverage collecting enable does not synchronize with the test case execution either.

In our second implementation, 2 helper functions `tested_driver_begin`, `tested_driver_end` are added. These functions enable and disable code coverage collecting respectively. The calls to these functions are manually inserted into our tested gadget driver to mark the start and end of a test case execution iteration. Then, the handlers of `ioctl` commands only need to wait for the corresponding helper to be executed before returning. With this implementation, the code coverage collecting is now synchronized with the test case execution and the code coverage determinism we observe is much better. The result will be discussed with more detail in Section 5.2

²Code coverage determinism: code coverage information output is the same providing the same input test case.

In this chapter, we present the result when performing fuzzing testing on mass storage gadget driver using our fuzzing system. Evaluation are performed on an Ubuntu 18.04.5 machine equipped with 2 CPUs and 7.5GB of RAM.

5.1 Execution speed

Running our fuzzer for 1 hour, approximately 48100 test cases are executed, the average execution speed is 13.4 exec/sec. This execution speed is lower than most normal user space program fuzzing speeds. However, compared with other USB driver fuzzers, it is significantly faster where the execution speed of Syzkaller and USBFuzz are 2.5 exec/sec and 2.6 exec/sec respectively [14]. The reason for this blazing fast speed is that we don't reset the USB connection in each testing iteration. With this approach, we reduce a lot of time loading and matching the host and gadget driver. However, not resetting USB connection also means that the testing environment for each test case may not be the same which results in the code coverage indeterminism. Moreover, it is harder to determine the begin and end of test case execution and manually adding 2 helper functions is required. Overall, the significant speedup of test case execution can be utilized to have a much more efficient fuzzing.

Table 5.1: Execution speed

	Our fuzzer	Syzkaller	USBFuzz
Execution speed (exec/sec)	13.4	2.5	2.6

5.2 Code coverage determinism

Code coverage information is the primary feedback for generating test cases so its correctness plays an important role. However, verifying the correctness of code coverage information is a difficult problem that requires a lot of debugging and testing. In this thesis, we use the code coverage determinism as an indicator for the incorrectness of code coverage information. The determinism of code coverage information requires our system to return the same code coverage information when the same test case is provided. This infers that the code coverage feedback represents the execution result of input test case and the environment variables if exist do not impact the end result.

We use the AFL fuzzer’s stability score to measure the code coverage determinism. The stability score represents the similarity percentage between code coverage information collected during multiple executions of an interesting test case.

During 1 hour running, we get the following stability scores

Table 5.2: Stability scores during 1 hour testing

	1	2	3	4	5	6	7	8	9	10
Stability score	100%	100%	100%	100%	100%	100%	71%	100%	48%	37%

As we can see, in some cases, the code coverage collected is not deterministic which is possibly due to the differences in execution environment or the incorrectness in our insertion of 2 helper functions. In the future work, we wish to find a better way to determine the execution interval and minimise the effect of environment variants. Generally, our current implementation still achieves an acceptable code coverage determinism in most cases.

6.1 Conclusion

In this thesis, we proposed and implemented a system for testing Android USB gadget drivers using fuzzing method. Our system is designed to be compatible with AFL fuzzer, a widely used fuzzer in academic research. Test cases generated by AFL fuzzer are delivered automatically to the USB gadget driver. Furthermore, our system is able to run in an emulated environment and requires no special hardware.

6.2 Future work

AFL fuzzer has its own limitations and it's best suited with tested programs that do not have complicated input and output interactions (e.g. program only receives input once then processes: file parser, image processing program). However, not all USB gadget drivers are that simple, some of them expect multiple stateful inputs which make it very complicated to test with AFL fuzzer. In the future, we wish to implement our own test case generator to overcome that limitation.

Besides, as stated in Section 4.4, 2 helper functions must be added manually to determine the begin and end of test case execution process in USB gadget fuzzer. We wish that this process can be automated. One solution for this problem is to find a signal in the kernel to determine this interval. Another approach is to use program analysis when compiling USB gadget driver to automatically identify the begin and end of the execution.

Appendices

User Interface

First, we need to build our testing environment as discussed in Section 4.2. Then, we add 2 helper functions `tested_driver_begin`, `tested_driver_end` to our tested USB gadget driver to define the begin and end of test case execution. In our host environment, we run `make manager` to compile the `Manager` and in QEMU environment `make executor` to compile the `Executor`. After that, we can provide some seed inputs into `input` directory, the variety of seed inputs can improve the fuzzing efficiency. Lastly, we edit the load command of USB gadget driver if necessary then run `run.sh` file to start our system.

Our system spins up the QEMU instance and starts AFL fuzzer

```
minh@minh:~/usb_fuzzer$ ./run.sh
core
Listening ...
/usr/bin/qemu-system-x86_64
/usr/bin/ssh
New connection
Start AFL
/home/minh/AFL/afl-fuzz
```

Figure A.1: Start-up process of our system

After that, we have the AFL fuzzer's UI which is familiar with researchers who have used it before. This UI provide the information about our fuzzing process such as execution speed, code coverage determinism, ...

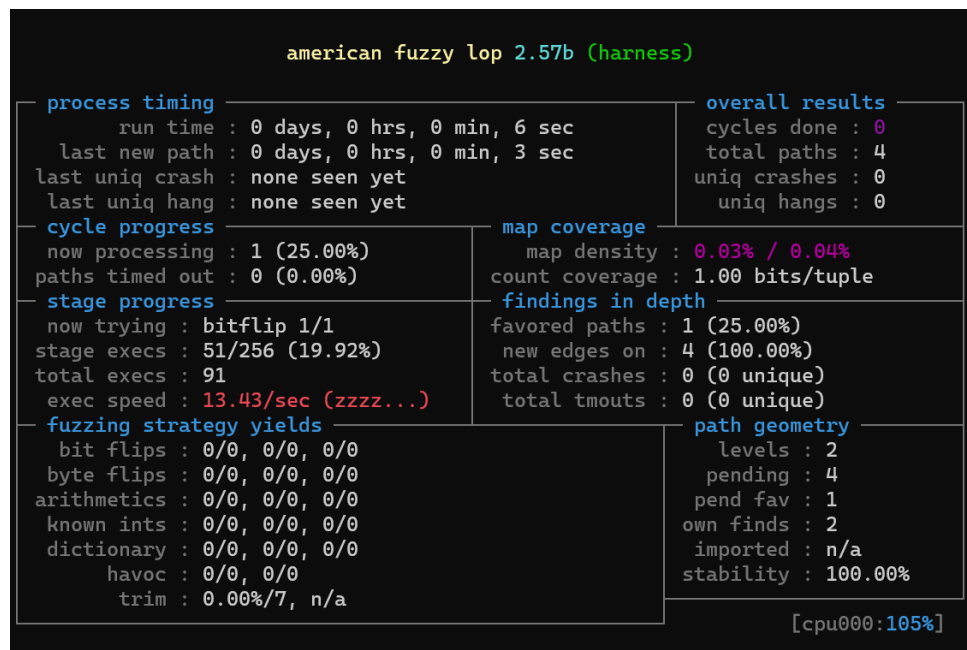


Figure A.2: AFL fuzzer's UI

Bibliography

- [1] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. “Klee: unassisted and automatic generation of high-coverage tests for complex systems programs.” In: *OSDI*. Vol. 8. 2008, pp. 209–224.
- [2] Android documentation. *Android Common Kernel*. URL: <https://source.android.com/devices/architecture/kernel/android-common> (visited on 07/21/2021).
- [3] GCC documentation. *GCC instrumentation options*. URL: <https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html> (visited on 07/21/2021).
- [4] Andrea Fioraldi et al. “AFL++ : Combining Incremental Steps of Fuzzing Research”. In: *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, Aug. 2020. URL: <https://www.usenix.org/conference/woot20/presentation/fioraldi>.
- [5] Patrice Godefroid, Michael Y. Levin, and David Molnar. “SAGE: Whitebox Fuzzing for Security Testing: SAGE Has Had a Remarkable Impact at Microsoft.” In: *Queue* 10.1 (Jan. 2012), pp. 20–27. ISSN: 1542-7730. DOI: 10.1145/2090147.2094081. URL: <https://doi.org/10.1145/2090147.2094081>.
- [6] Google. *Linux kernel bugs found by Syzkaller*. URL: https://github.com/google/syzkaller/blob/master/docs/linux/found_bugs.md (visited on 07/21/2021).
- [7] Google. *Syzkaller Github Repository*. URL: <https://github.com/google/syzkaller> (visited on 07/21/2021).
- [8] Google. *USB bugs found by Syzkaller*. URL: https://github.com/google/syzkaller/blob/master/docs/linux/found_bugs_usb.md (visited on 07/21/2021).
- [9] NCC Group. *umap2 Github Repository*. URL: <https://github.com/nccgroup/umap2> (visited on 07/21/2021).
- [10] Linux kernel documentation. *Adding a New System Call*. URL: <https://www.kernel.org/doc/html/v4.14/process/adding-syscalls.html> (visited on 07/21/2021).

- [11] LLVM Project. “libFuzzer – a library for coverage-guided fuzz testing”. In: (2018). URL: <https://llvm.org/docs/LibFuzzer.html> (visited on 07/21/2021).
- [12] Michal Zalewski. “American Fuzzy Lop”. In: (2017). URL: <https://lcamtuf.coredump.cx/afl/> (visited on 07/21/2021).
- [13] Michal Zalewski. “American Fuzzy Lop Whitepaper”. In: (2017). URL: https://lcamtuf.coredump.cx/afl/technical_details.txt (visited on 07/21/2021).
- [14] Hui Peng and Mathias Payer. “USB fuzz: A Framework for Fuzzing {USB} Drivers by Device Emulation”. In: *29th {USENIX} Security Symposium ({USENIX} Security 20)*. 2020, pp. 2559–2575.
- [15] Sergej Schumilo, Ralf Spennberg, and Hendrik Schwartke. “Don’t trust your USB! How to find bugs in USB device drivers”. In: *Blackhat Europe* (2014).